

Задание 1.1

Обзор среды разработки GoLand

Введение

В обзоре рассматриваются назначение GoLand, требования к рабочему месту и основные возможности среды: создание проекта, написание и форматирование кода, запуск, компиляция, отладка, контроль версий и AI-функции. Примеры приведены для GoLand 2026.2, Windows 11 x64 и Go SDK 1.27.1; интерфейс среды — английский.

1. Характеристика GoLand и начало разработки — Качков К. Е.

1.1 Назначение продукта

GoLand — интегрированная среда разработки компании JetBrains для программирования на языке Go. Основная аудитория — разработчики серверных приложений, сетевых сервисов и утилит, а также студенты, изучающие Go. В одной программе объединены редактор, анализ кода, средства запуска и отладки, терминал и работа с системой контроля версий. [P1-1]

Помимо Go, продукт поддерживает JavaScript и TypeScript, работу с базами данных, Docker и Git. Поэтому разработчик может редактировать код сервиса, проверять связанные файлы и работать с репозиторием из одной среды. Для данного конспекта выбрана версия GoLand 2026.2. [P1-1]

GoLand распространяется по лицензии JetBrains. Для знакомства предусмотрен бесплатный пробный период до 30 дней; для коммерческой работы — платная подписка. Студенты и преподаватели могут получить бесплатный доступ по образовательной программе при соблюдении её условий. Доступную лицензию можно активировать через Help → Register и JetBrains Account. [P1-2]

1.2 Применение в организации практики

Практика связана с кафедрой информационных технологий и электронного обучения РГПУ им. А. И. Герцена. В задания входят изучение средств разработки, автоматизация установки программ в компьютерных аудиториях, управление проектами и работа с Git. Поэтому GoLand рассматривается применительно к учебной разработке программ и утилит на Go.

Например, при подготовке учебной утилиты студент создаёт проект, пишет функции, проверяет их поведение и сохраняет изменения в Git. Преподавателю или другому участнику группы можно передать исходный код и историю изменений через GitHub. GoLand обеспечивает рабочую часть этого процесса: редактирование, навигацию по коду, запуск и поиск ошибок. Организация заданий и итоговая публикация материалов выполняются в соответствующих сервисах.

Для такой работы полезны подсказки редактора и переходы между объявлениями и вызовами функций: они помогают разбираться в незнакомом проекте. В рамках практики GoLand анализируется как выбранный инструмент разработки; конкретный сценарий ниже — небольшой учебный проект.

1.3 Аппаратные и программные требования

Для установки и работы самой IDE в документации приведены следующие требования. [P1-3]

- Процессор архитектуры x86_64 или ARM64 с четырьмя ядрами.
- Оперативная память — 8 ГБ всего; 3 ГБ должны быть доступны процессам IDE.
- Свободное место на диске — 10 ГБ.
- Разрешение экрана — не менее 1280 × 720.

— Поддерживаемые версии Windows — 10 и 11. В обзоре используется Windows 11 x64.

Отдельно устанавливать Java для запуска GoLand не требуется: JetBrains Runtime поставляется вместе со средой. Место для проектов, зависимостей и результатов сборки нужно учитывать дополнительно. [P1-3]

Для разработки нужен Go SDK: в нём находятся компилятор, стандартная библиотека и команда go. В плане работы выбрана версия 1.27.1. Для локального контроля версий нужен Git, а для размещения проекта на GitHub — учётная запись сервиса и доступ к сети. Git и GitHub не требуются для простого редактирования файла, но нужны на соответствующих этапах совместной работы. [P1-4]

1.4 Установка и создание проекта

С официального сайта JetBrains необходимо загрузить установщик GoLand для Windows x64, запустить файл .exe и пройти шаги мастера. После установки среда запускается через меню «Пуск». При первом запуске выбирается доступный способ активации или пробный период. [P1-2, P1-3]

В Settings (Ctrl+Alt+S) можно выбрать тему и размер шрифта редактора. В разделе Go → GOROOT задаётся Go SDK: Add SDK → Local позволяет указать уже установленный SDK, Add SDK → Download — загрузить нужную версию. GOROOT указывает на корневую папку SDK, содержащую каталоги bin и src, а не на сам файл go.exe. [P1-4]

Создание проекта goland-demo:

1. На стартовом экране нажать New Project; в открытой IDE — File → New → Project.
2. Выбрать Go, указать папку goland-demo и проверить выбранный Go SDK 1.27.1 в поле GOROOT.
3. Нажать Create. Для нового проекта используется интеграция с Go Modules.
4. В окне Project открыть контекстное меню папки проекта и выбрать New → Go File. Задать имя main и добавить код из пункта 1.5.
5. Проверить, что в корне проекта есть go.mod и main.go. Если модуль ещё не создан, выполнить в терминале из этой папки команду go mod init example.com/goland-demo. [P1-4, P1-9]

Go Modules — механизм объединения пакетов в модуль и управления его зависимостями. Файл go.mod содержит путь модуля, минимальную версию Go и требования к зависимостям. Исходный код находится в файлах с расширением .go. Для учебного проекта достаточно следующего go.mod; сторонние библиотеки здесь не используются. [P1-9]

```
module example.com/goland-demo

go 1.27.1
```

Итог этого этапа — проект с выбранным SDK, файлом описания модуля и исходным файлом main.go. Его можно открыть в GoLand и продолжить работу средствами редактора.

1.5 Интерфейс и написание кода

Окно Project показывает папки и файлы. В центре расположен редактор с вкладками открытых файлов. Верхняя панель содержит меню, выбор проекта и ветки Git, а также конфигурации запуска. Окна Run, Debug и Terminal предназначены соответственно для вывода программы, отладки и выполнения команд. Окна инструментов можно перемещать или открывать через View → Tool Windows. [P1-5]

Для демонстрации возможностей редактора используется файл main.go:

```
package main

import "fmt"

func sum(a, b int) int {
    return a + b
}
```

```
func                               main()                               {
                                result := sum(2, 3)
                                fmt.Println(result)
}
```

Функция `sum` принимает два целых числа и возвращает их сумму. Функция `main` — точка входа в программу. В данном примере `result` получает значение 5, а `fmt.Println` выводит его. Импорт `fmt` подключает пакет стандартной библиотеки для форматированного ввода и вывода.

Автодополнение. В строке вывода ввести `fmt.Pr` и нажать `Ctrl+Space`. Среда предложит подходящие имена, в том числе `Println`. Выбор подсказки дополняет имя функции и сокращает ручной набор. [P1-6]

Справка по функции. Поставить курсор на `Println` и нажать `Ctrl+Q`. В окне `Quick Documentation` отображаются сигнатура и описание функции; это помогает проверить назначение и параметры без выхода из редактора. [P1-6]

Переход к определению. Удерживая `Ctrl`, щёлкнуть по `sum` в вызове `sum(2, 3)`. Редактор перейдёт к объявлению функции. Если нужен только быстрый просмотр определения, использовать `Ctrl+Shift+I`. На большом проекте эти действия избавляют от ручного поиска нужного файла. [P1-7]

Поиск использований. Поставить курсор на имя `sum` в её объявлении и нажать `Alt+F7`. В окне `Find` появится место вызова в `main`. Перед изменением функции так можно проверить, какой код зависит от неё. [P1-8]

Эти операции показывают различие между возможностями редактора: автодополнение помогает написать выражение, справка объясняет его назначение, переход открывает реализацию, а поиск использований показывает связи внутри проекта.

2. Форматирование, запуск, компиляция и отладка — Момот М.-М. М.

2.1. Форматирование кода, управление импортами, предупреждения IDE и быстрые исправления.

Среда разработки `GoLand` использует статический анализатор кода, работающий с консольными утилитами форматирования `gofmt` и обновления импортируемых пакетов `goimports`. `GoLand` выполняет выравнивание синтаксических конструкций. Для принудительного форматирования всего файла или выделенного фрагмента применяется сочетание клавиш `Ctrl + Alt + L`. [P2-1]

При вызове в коде функции или структуры из стандартной библиотеки или внешнего модуля IDE добавляет соответствующий пакет в секцию импортов. При удалении строк, использующих пакет, неиспользуемый импорт вырезается из файла. В Go наличие объявленных, но неиспользуемых импортов запрещено спецификацией и приводит к ошибке компиляции. Автоматизацию этих операций можно закрепить за каждым сохранением файла, установить настройки `Reformat code` и `Optimize imports`. [P2-2]

Проверки кода в `GoLand` постоянно сканируют исходный код. Синтаксические ошибки выделяются красным цветом, логические предупреждения - желтым, стилистические замечания - серым цветом. При обнаружении дефекта курсор устанавливается на подсвеченное слово и используется комбинация `Alt + Enter`, далее нужно выбрать автоматическое исправление. [P2-3]

Пример кода до исправления

В этом коде есть нарушения отступов, подключён неиспользуемый пакет `math` и объявлена переменная `age`, которая не задействована в вычислениях.

```
package main
import (
    "fmt"
    "math"
)
func main() {
    age := 20
    fmt.Println("Hello, World!")
}
```

Использование команды `Ctrl + Alt + L` выравнивает отступы внутри функции `main`. С помощью комбинации `Alt + Enter` на строке пакета «`math`» вызывается контекстное меню, где выбирается опция оптимизации импортов, лишняя строка удаляется. На строке с переменной `age` быстрое исправление удаляет неиспользуемый идентификатор. После применения автоматических инструментов исходный код приводится к синтаксически корректному виду.

```
package main
import "fmt"
func main() {
    fmt.Println("Hello, World!")
}
```

2.2. Запуск программы.

Процесс выполнения программ в GoLand контролируют конфигурации запуска. Среда автоматически распознает файлы, содержащие точку входа `func main()`, и позволяет запускать их без обращения к терминалу. Для быстрого запуска используется элемент на левом поле текстового редактора напротив функции `main`. Нажатие на данную иконку или использование сочетания клавиш `Ctrl + Shift + F10` создает временную конфигурацию и выполняет текущий код. [P2-4]

Настройка постоянной конфигурации необходима при передаче аргументов командной строки или переменных окружения. Конфигурация настраивается через меню путем добавления шаблона `Go Build`. При редактировании конфигурации необходимо отметить `Run kind` для определения объекта компиляции. Значение `File` запускает один файл, а значение `Directory` собирает пакет в указанной папке. Нужно также отметить `Program arguments`, строку параметров, передаваемых в программу. Параметры разделяются пробелами. Также следует отметить `Environment`, таблицу переменных окружения в формате «ключ-значение», доступных через функции стандартной библиотеки. [P2-5]

Запуск конфигурации выполняется нажатием кнопки со стрелкой на панели инструментов или комбинацией `Shift + F10`. При запуске в нижней части интерфейса открывается панель `Run`, отображающая команду сборки, вывод стандартного потока, код завершения процесса. [P2-5]

Если программа завершается со сбоем, строки вызова в консоли становятся кликабельными ссылками с путями к файлам и номерами строк кода. Переход по ссылке перемещает курсор в редакторе к строке, вызвавшей сбой. Кнопка `Stop` либо сочетание `Ctrl + F2` позволяет принудительно завершить процесс. [P2-6]

Пример программы

```
package main
import (
    "fmt"
    "os"
)
func main() {
    args := os.Args[1:]
    if len(args) == 0 {
        fmt.Println("No arguments provided")
        return
    }
    fmt.Println("First argument:", args[0])
}
```

Вывод программы в окне Run при указании аргумента Test:

```
C:\Users\User\AppData\Local\Programs\Go\bin\go.exe    run    C:\Users\User\GoLandProjects\GoLand-
demo\main.go Test
First argument: Test
Process finished with exit code 0
```

Выполнение программы начинается с функции `main`, которая считывает параметры командной строки в срез `args`. Встроенная функция `len` определяет количество элементов в этом срезе, а с помощью конструкции `if` проверяется длина среза, если аргументы отсутствуют, выводится сообщение «No arguments provided», работа завершается. В противном случае функция `fmt.Println` принимает первый элемент и выводит его значение в консоль.

2.3. Компиляция.

При запуске программы внутри среды разработки создается временный исполняемый файл в скрытом каталоге операционной системы. Он автоматически удаляется после завершения процесса. Этот режим предназначен для тестирования в процессе написания кода. [P2-7]

Компиляция переводит исходный код в постоянный исполняемый файл, содержащий машинный код. Полученный файл можно запускать на других системах без установки GoLand и инструментов языка Go. [P2-7]

Для сборки исполняемого файла используется встроенный терминал, вызываемый комбинацией клавиш `Alt + F12`. В командной строке вводится стандартная команда: `go build -o program.exe main.go`. Параметр `-o` позволяет задать имя файла. Если выполнить команду без дополнительных параметров, компилятор соберет проект и поместит исполняемый файл в корневой каталог текущего проекта под именем этой папки. В дереве файлов проекта отобразится элемент с расширением `.exe`. [P2-5]

2.4. Отладка.

Если программа работает без системных ошибок, но выдает неверный результат вычислений, используется интерактивная отладка. Она позволяет временно приостановить выполнение приложения в заданной строке кода, проанализировать состояние памяти и проследить за выполнением программы с помощью встроенного отладчика. [P2-8]

Точка останова указывает на приостановление выполнения до заданной строки кода. Устанавливается кликом по полю слева от номеров строк в редакторе. Шаг с обходом от текущей строки перемещает указатель на следующую, не заходя внутрь вызываемых функций. Шаг с

обходом выполняется нажатием клавиши F8. Шаг со входом помогает перейти внутрь вызываемой функции для анализа ее логики. Вызывается клавишей F7. [P2-6]

Пример исправления логической ошибки

В данном коде функция вычисляет среднее арифметическое элементов массива. Функция возвращает некорректно округленный результат.

```
package main
import "fmt"
func average(numbers []int) float64 {
    total := 0
    for i := 0; i < len(numbers); i++ {
        total += numbers[i]
    }
    return float64(total / len(numbers))
}
func main() {
    numbers_list := []int{10, 20, 15, 40}
    result := average(numbers_list)
    fmt.Printf("Результат: %f\n", result)
}
```

Нужно установить точку останова на строке с вычислением среднего значения. затем запустить отладку комбинацией клавиш Shift + F9 или кнопкой Debug. При достижении точки останова необходимо проверить сумму элементов, длину массива, промежуточный результат деления в панели Variables. Затем с помощью шагов отладки с использованием клавиш F8 и F7 нужно проверить изменение значений на каждом этапе вычислений. В данном коде выражение `return float64(total/len(numbers))` приводило к ошибке вычислений из-за деления нацело, после этого результат преобразовывался в вещественный тип. В исправленном коде выражение `return float64(total)/float64(len(numbers))` приводит операнды к типу float64, деление выполняется без потери точности.

```
package main
import "fmt"
func average(numbers []int) float64 {
    total := 0
    for i := 0; i < len(numbers); i++ {
        total += numbers[i]
    }
    return float64(total) / float64(len(numbers))
}
func main() {
    numbers_list := []int{10, 20, 15, 40}
    result := average(numbers_list)
    fmt.Printf("Результат: %f\n", result)
}
```

После внесения изменений программа выводит результат 21.250000.

3. Контроль версий, AI и оценка продукта - Парванян Г.С.

3.1. Работа с Git в GoLand

GoLand имеет встроенную поддержку Git, которая активируется автоматически после установки системы контроля версий на компьютер. Плагин Git включён по умолчанию и не требует отдельной установки. [P3-1]

При создании нового проекта в GoLand можно сразу выбрать опцию "Git репозиторий" в мастере создания проекта. GoLand автоматически определяет наличие Git на компьютере: если исполняемый файл не найден, IDE предложит его загрузить. После создания проекта с Git-репозиторием становятся доступны специализированные инструменты: виджет VCS для управления ветками, окно Commit для работы с локальными изменениями и окно Git для работы с историей.

Если проект уже существует без Git, репозиторий можно создать через меню VCS => Create Git Repository.

Для файлов, которые не должны попадать под контроль версий, GoLand позволяет быстро добавить их в .gitignore. Достаточно щёлкнуть правой кнопкой мыши по файлу и выбрать "Add to .gitignore". Важно учитывать, что .gitignore игнорирует только новые, ещё не отслеживаемые файлы: если файл уже был закоммичен, Git продолжит его отслеживать даже после добавления в .gitignore. [P3-2]

GoLand предоставляет несколько способов отслеживания изменений. В окне Commit отображаются все модифицированные файлы (выделены синим), новые файлы, добавленные в Git, но ещё не закоммиченные (зелёным), и неотслеживаемые файлы (Unversioned Files). В редакторе изменения подсвечиваются маркерами на полях рядом с изменёнными строками с цветовой кодировкой: модифицированные строки, строки с изменёнными пробелами и добавленные строки.

Для создания коммита необходимо открыть окно Commit, выбрать файлы для включения в коммит, ввести сообщение и нажать Commit. Просмотр истории осуществляется через вкладку Log окна Git. В Log отображаются все коммиты всех ветвей и удалённых репозиториев, доступен поиск по сообщениям коммитов, хешам и регулярным выражениям, а также фильтрация по ветке, пользователю и дате. [P3-3]

3.2. Публикация в удалённом репозитории

GoLand поддерживает интеграцию с GitHub через отдельный плагин, который также включён по умолчанию. Для подключения необходимо авторизоваться через GitHub-аккаунт или Personal Access Token. В окне "Get from Version Control" можно выбрать GitHub в качестве источника и войти в систему. При включённой двухфакторной аутентификации потребуется ввести код подтверждения. [P3-4]

Самый быстрый способ опубликовать проект - использовать функцию "Share Project on GitHub" (VCS => Import into Version Control => Share Project on GitHub). Эта команда выполняет инициализацию репозитория, создание коммита, привязку удалённого репозитория и первую отправку изменений в одном действии. При первой публикации GoLand предложит ввести имя репозитория и описание.

После создания коммита изменения можно отправить в удалённый репозиторий через Push (Ctrl+Shift+K или через меню Git => Push). Для получения изменений от других участников используется Pull (Git => Pull). Перед Push рекомендуется выполнить Pull, чтобы убедиться, что локальная копия актуальна. [P3-5]

3.3. AI-функции в GoLand

В GoLand используется плагин JetBrains AI Assistant, который предоставляет доступ к возможностям искусственного интеллекта. Плагин не входит в стандартную поставку GoLand и не включён по умолчанию. Для активации необходимо:

- Установить плагин AI Assistant.
- Получить лицензию JetBrains AI Service.
- Явно согласиться с условиями использования JetBrains AI Service и политикой допустимого использования.

Без выполнения этих условий AI Assistant не активируется и не получает доступ к коду пользователя.

AI Assistant позволяет выделить фрагмент кода, открыть окно AI и задать вопрос, например: "What does this code do?" - инструмент предоставит объяснение на естественном языке. Для предложения исправления можно использовать команды вроде "Fix this error" или "Suggest a fix", и AI предложит вариант изменения кода с обоснованием. [P3-6]

AI Assistant не работает без установленного плагина и действующей лицензии. Даже при наличии лицензии доступ может быть ограничен территориально. [P3-7]

3.4. Заключение

GoLand - это специализированная среда разработки для языка Go от JetBrains, обладающая лучшей на рынке системой статического анализа для Go. IDE понимает интерфейсы, что экономит десятки часов на поиске ошибок компиляции. Встроенная интеграция с Git и GitHub позволяет выполнять весь цикл работы с версиями без переключения в терминал. Инструменты рефакторинга, отладки и навигации по коду работают интегрированно и не требуют сложной настройки плагинов.

Источники раздела 1 — Качков К. Е.

Дата обращения к электронным источникам — 25.09.2026.

[P1-1]	JetBrains.	GoLand	—	описание	продукта.
https://www.jetbrains.com/go/					
[P1-2]	JetBrains.		Register		GoLand.
https://www.jetbrains.com/help/go/register.html					
[P1-3]	JetBrains.		Install		GoLand.
https://www.jetbrains.com/help/go/installation-guide.html					
[P1-4]	JetBrains.	Go	—	создание проекта и настройка	SDK.
https://www.jetbrains.com/help/go/create-a-project-with-go-modules-integration.html					
[P1-5]	JetBrains.		New		UI.
https://www.jetbrains.com/help/go/new-ui.html					
[P1-6]	JetBrains.		Code		completion.
https://www.jetbrains.com/help/go/auto-completing-code.html					
[P1-7]	JetBrains.	Definition	and	type	definition.
https://www.jetbrains.com/help/go/viewing-definition.html					
[P1-8]	JetBrains.	Search		for	usages.
https://www.jetbrains.com/help/go/find-highlight-usages.html					
[P1-9]	The	Go	Authors.	go.mod	file
https://go.dev/doc/modules/gomod-ref					

Источники раздела 2 — Момот М.-М. М.

Дата обращения к электронным источникам — 25.09.2026.

[P2-1] JetBrains. Go tools.

<https://www.jetbrains.com/help/go/integration-with-go-tools.html>

[P2-2] JetBrains. Auto import.

<https://www.jetbrains.com/help/go/creating-and-optimizing-imports.html>

[P2-3] JetBrains. Inspections.

<https://www.jetbrains.com/help/go/inspections-settings.html>

[P2-4] JetBrains. Running.

<https://www.jetbrains.com/help/go/running-applications.html>

[P2-5] JetBrains. Go Build. <https://www.jetbrains.com/help/go/go-build.html>

[P2-6] JetBrains. Step through the program.

<https://www.jetbrains.com/help/go/stepping-through-the-program.html>

[P2-7] JetBrains. Run/debug configurations. <https://www.jetbrains.com/help/go/run-debug-configuration.html>

[P2-8] JetBrains. Examine suspended program. <https://www.jetbrains.com/help/go/examining-suspended-program.html>

Источники	раздела	3	-	Парванян	Г.С
Дата	обращения	к	электронным	источникам	— 25.09.2026.
[P3-1]	Add files to	Git	and track changes	- GoLand	documentation.
	https://www.jetbrains.com/help/go/adding-files-to-version-control.html				
[P3-2]	Investigate changes	in	Git repository	- GoLand	documentation.
	https://www.jetbrains.com/help/go/investigate-changes.html				
[P3-3]	Tutorial: Getting started	with	Git in GoLand	- GoLand	documentation.
	https://www.jetbrains.com/help/go/working-with-git-tutorial.html				
[P3-4]	Manage projects	hosted	on GitHub	- GoLand	documentation.
	https://www.jetbrains.com/help/go/manage-projects-hosted-on-github.html				
[P3-5]	Sync with a remote	Git	repository	- GoLand	documentation.
	https://www.jetbrains.com/help/go/sync-with-a-remote-repository.html				
[P3-6]	AI Assistant in	JetBrains	IDEs	- GoLand	documentation.
	https://www.jetbrains.com/help/go/ai-assistant-in-jetbrains-ides.html				
[P3-7]	JetBrains AI plans	and	usage	- AI Assistant	documentation.
	https://www.jetbrains.com/help/go/ai-assistant/licensing-and-subscriptions.html				